

Shell Scripting

ROADMAP

#1

Everything Every DevOps Engineer
Must Know

In this series you will learn:

- ✓ Basics to Advanced Concepts
- ✓ Practical Examples
- ✓ Real-world DevOps Use Cases
- ✓ Best Practices
- ✓ Interview Questions



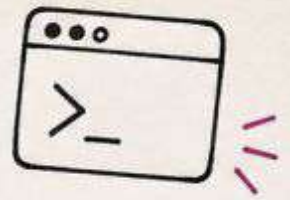
Small Scripts,
Big Impact!

Automate
Simplify
Excel



Swipe →

2. WHAT IS SHELL SCRIPTING?



DEFINITION

A shell script is a text file containing a series of Linux commands that are executed by the shell (such as Bash).

WHY SHELL SCRIPTING?

- Automates repetitive tasks
- Saves time and effort
- Reduces human errors
- Easy to write and execute
- Powerful for system administration

ADVANTAGES

- ✓ Automation
- ✓ Consistency
- ✓ Efficiency
- ✓ Flexibility
- ✓ Integrates with Linux tools
- ✓ Easy to learn and maintain

WHERE IT'S USED?

- System Administration
- DevOps Automation
- Backup & Monitoring
- Log Analysis
- CI/CD Pipelines
- Deployment Scripts
- Cloud Operations



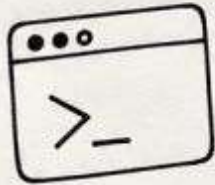
COMMON SHELLS

Bash  (Default in most Linux)	Sh \$ (Bourne Shell)	Zsh \$ (Z Shell)	Ksh \$ (Korn Shell)	Csh \$ (C Shell)
---	-----------------------------------	-------------------------------	----------------------------------	-------------------------------



Most widely used: Bash

Swipe →



3. SHELL SCRIPT BASICS



1 SHEBANG (#!/bin/bash)

- The first line of the script. Tells the system which shell to use to run the script.



Example :
#!/bin/bash

Always start your script with shebang!

2 SCRIPT STRUCTURE

- A script is a series of commands written in a file.
- General structure:

```
#!/bin/bash
# 1. Comments / Description
# 2. Variables
# 3. Commands
# 4. Logic (if, for, while, etc.)
# 5. Exit
```

Example Script :

```
#!/bin/bash
# This script prints Hello
NAME="DevOps"
echo "Hello $NAME"
exit 0
```

3 COMMENTS

- Used to explain the code.
- Ignored by the shell.
- Helps in readability.

Example :

```
# This is a single line comment
# You can explain what your script is doing here.
```

Good comments make your script easy to understand!



4 EXECUTE PERMISSIONS

- Give execute permission to the script file.
- Without this, script won't run.

Command :

```
chmod +x script.sh
```



Makes the file executable.

5 RUNNING SCRIPTS

- Once the script has execute permission, you can run it.
- Use either of the methods.

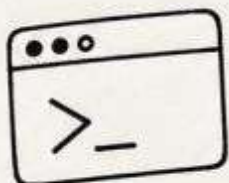
Ways to run :

1. ./script.sh # Run using relative path
2. bash script.sh # Run using bash explicitly



Master these basics and you're ready to automate the world! 😊

Swipe →



4. VARIABLES



Variables are used to store data values.

1 CREATING VARIABLES

- No spaces around = sign.

Example :

```
NAME = "DevOps"  
AGE = 41  
CITY = "Bengaluru"
```

4 EXPORT VARIABLES

- Makes the variable available to child processes and subshells.

Example :

```
export PATH=$PATH:/opt/bin  
echo $PATH
```

export
is used
to share
variables!



2 ACCESSING VARIABLES

- Use \$ sign to access the value.

Example :

```
echo $NAME  
echo "Hello $NAME"  
echo "Age is $AGE"
```

5 ENVIRONMENT VARIABLES

- Predefined variables available in the system.

Common Examples :

```
$PATH → Directories in executable path  
$HOME → Home directory  
$USER → Current username  
$SHELL → Current shell  
$PWD → Present working directory
```



3 READ-ONLY VARIABLES

- Once set, value cannot be changed.

Example :

```
readonly ID=101  
ID=202 # Will throw error
```

6 NAMING CONVENTIONS

- ✓ Start with a letter or underscore (_)
- ✓ Can contain letters, numbers and underscore
- ✓ No spaces or special characters
- ✓ Case sensitive (myVar and myvar are different)
- ✓ Use meaningful and descriptive names

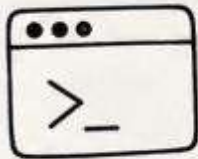


Use meaningful variable
names to make your
scripts easy to read
and maintain! 😊



Understanding variables is the first step to
writing powerful shell scripts! 😊

Swipe →



5. USER INPUT



User input allows scripts to interact with users.

1 READ COMMAND

- The read command is used to take input from the user.

Example :

```
read NAME
echo "Hello $NAME"
```



What is your name?

2 PROMPT MESSAGES

- Show a message before taking input.

Example :

```
read -p "Enter your name: " NAME
echo "Hello $NAME"
```

Always give a clear prompt to users!

3 HIDDEN PASSWORD INPUT (read -s)

- Use read -s to hide input on the screen (e.g., passwords).

Example :

```
read -s -p "Enter your password: " PASS
echo -e "\nPassword saved!"
```



4 DEFAULT VALUES

- Provide default values in case user enters nothing.

Example :

```
read -p "Enter your age [18]: " AGE
AGE=${AGE:-18} # If AGE is empty, set default to 18
echo "Your age is $AGE"
```

If user just presses Enter, default value will be used.

5 INPUT VALIDATION

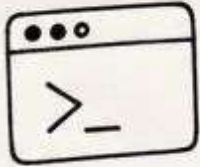
- Always validate user input to avoid errors.

Example :

```
read -p "Enter your age: " AGE
while ! [[ "$AGE" =~ ^[0-9]+$ ]]; do
    echo "Please enter a valid number."
    read -p "Enter your age: " AGE
done
```



Good user input handling makes your scripts smart, safe and user-friendly! 😊



6. COMMAND LINE ARGUMENTS



Arguments are passed to a script from the command line.

EXAMPLE

```
$ ./myscript.sh devops linux scripting
```

↓ ↓ ↓ ↓

\$0 \$1 \$2 \$3

These special variables help you access the arguments passed to your script.



PARAMETER	DESCRIPTION	EXAMPLE (using above)
\$0	The name of the script.	echo \$0 → ./myscript.sh
\$1	The first argument passed to the script.	echo \$1 → devops
\$2	The second argument.	echo \$2 → linux
\$#	The number of arguments passed to the script.	echo \$# → 3
\$@	All arguments passed to the script (as separate words).	echo \$@ → devops linux scripting
\$*	All arguments passed to the script (as a single word).	echo \$* → devops linux scripting (as one word)
shift	Shifts positional parameters to the left.	<u>shift</u> # removes \$1 echo \$1 → linux # devops is removed echo \$2 → scripting

QUICK DEMO

```
$ ./myscript.sh  
devops linux scripting
```



```
echo $0 → ./myscript.sh  
echo $1 → devops  
echo $2 → linux  
echo $3 → scripting  
echo $# → 3
```

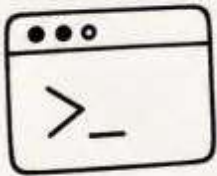


Use these parameters to make your scripts more dynamic and reusable!



Master command line arguments - your scripts will become more powerful! 😊

Swipe →



7. CONDITIONAL STATEMENTS



Conditional statements are used to make decisions in a script.

Use conditions to control the flow of your script! 😊

1 IF STATEMENT

- Executes a block of code if condition is true.

```
if [ condition ]; then
    # commands
fi
```

2 IF-ELSE STATEMENT

- Executes one block if condition is true, another if false.

```
if [ condition ]; then
    # commands if true
else
    # commands if false
fi
```

3 ELIF STATEMENT

- Used to check multiple conditions.

```
if [ condition1 ]; then
    # commands
elif [ condition2 ]; then
    # commands
else
    # default commands
fi
```

4 NESTED IF

- An if statement inside another if.

```
if [ condition1 ]; then
    if [ condition2 ]; then
        # commands
    else
        # commands
    fi
else
    # commands
fi
```

5 LOGICAL OPERATORS

- Combine multiple conditions.

Operator	Meaning	Example
-a	AND	[A -a B]
-o	OR	[A -o B]
!	NOT	[! A]

Examples:

```
if [ $AGE -gt 18 -a $AGE -lt 60 ]; then
    echo "Adult"
fi
```

```
if [ $USER = "root" -o $USER = "admin" ]; then
    echo "Welcome Admin"
fi
```

6 TEST COMMAND []

- Used to evaluate expressions and conditions.

Common Usage:

- ☒ Numeric comparison
- ☒ String comparison
- ☒ File tests
- ☒ Logical operations

Examples:

```
[ $AGE -gt 18 ]
[ "$NAME" = "DevOps" ]
[ -f /etc/passwd ]
[ -n "$USER" ]
```

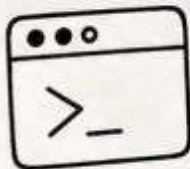
Tips 😊

Always use spaces inside
[and] like this:
[condition]
Not like this: [condition]



Master conditions, and you master the flow of your script! 😊

Swipe →



8. COMPARISON OPERATORS



Comparison operators are used to compare values in conditions and expressions.

Used inside test command [] or [[]] in shell scripting!



1 NUMERIC COMPARISON OPERATORS

Operator	Meaning	Example	Description
-eq	Equal	[\$a -eq \$b]	True if \$a is equal to \$b
-ne	Not Equal	[\$a -ne \$b]	True if \$a is not equal to \$b
-gt	Greater Than	[\$a -gt \$b]	True if \$a is greater than \$b
-lt	Less Than	[\$a -lt \$b]	True if \$a is less than \$b
-ge	Greater Than or Equal	[\$a -ge \$b]	True if \$a is greater than or equal to \$b
-le	Less Than or Equal	[\$a -le \$b]	True if \$a is less than or equal to \$b

Example:

a=10

b=20

[\$a -eq \$b] → False

[\$a -ne \$b] → True

[\$a -gt \$b] → False

[\$a -lt \$b] → True

[\$a -ge \$b] → False

[\$a -le \$b] → True

2 STRING COMPARISON OPERATORS

Operator	Meaning	Example	Description
=	Equal	["\$a" = "\$b"]	True if \$a is equal to \$b
!=	Not Equal	["\$a" != "\$b"]	True if \$a is not equal to \$b
-z	String is Empty	[-z "\$a"]	True if \$a has zero length (empty)
-n	String is Not Empty	[-n "\$a"]	True if \$a has non-zero length

Example:

name="DevOps"

empty=""

["\$name" = "DevOps"] → True

["\$name" != "Linux"] → True

[-z "\$empty"] → True

[-n "\$name"] → True

[-z "\$name"] → False

Note:

For string comparison, always use quotes around variables!



Usage:

- [expression] → For basic test
- [[expression]] → For advanced test (Recommended)

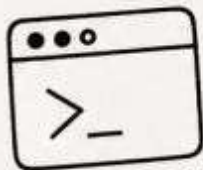
Tip:

Use spaces inside [] like this: [condition]
Not like this: [condition]



Master comparison operators - they are the foundation of smart decisions in scripts! 😊

Swipe →



9. LOOPS



Loops are used to execute a block of code multiple times until a certain condition is met.

Loops help you automate repetitive tasks efficiently!



① FOR LOOP

- Best when number of iterations is known.

Syntax:

```
for var in item1 item2 item3 ...  
do  
    commands  
done
```

Example:

```
for i in 1 2 3 4 5  
do  
    echo "Number: $i"  
done  
# Output: Number: 1 2 3 4 5
```

② WHILE LOOP

- Repeats while a condition is true.

Syntax:

```
while [ condition ]  
do  
    commands  
done
```

Example:

```
count=1  
while [ $count -le 5 ]  
do  
    echo "Count: $count"  
    count=$((count+1))  
done  
# Output: Count: 1 2 3 4 5
```

③ UNTIL LOOP

- Repeats until a condition becomes true.

Syntax:

```
until [ condition ]  
do  
    commands  
done
```

Example:

```
count=1  
until [ $count -gt 5 ]  
do  
    echo "Count: $count"  
    count=$((count+1))  
done  
# Output: Count: 1 2 3 4 5
```

④ BREAK

- Terminates the loop immediately.

Example:

```
for i in 1 2 3 4 5  
do  
    if [ $i -eq 3 ]; then  
        break  
    fi  
    echo "$i"  
done  
# Output: 1 2
```

break
stops the
loop!



⑤ CONTINUE

- Skips the remaining commands in the current iteration and continues with next iteration.

Example:

```
for i in 1 2 3 4 5  
do  
    if [ $i -eq 3 ]; then  
        continue  
    fi  
    echo "$i"  
done  
# Output: 1 2 4 5
```

continue
skips to the
next iteration!



⑥ NESTED LOOPS

- A loop inside another loop.

Example:

```
for i in 1 2  
do  
    for j in A B  
    do  
        echo "i=$i, j=$j"  
    done  
done  
# Output:  
# i=1, j=A i=1, j=B i=2, j=A i=2, j=B
```

Nested loops
are powerful
for handling
complex tasks!



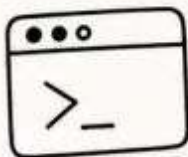
QUICK REFERENCE

for → Fixed number of iterations
while → Repeat while condition is true
until → Repeat until condition is true
break → Exit the loop
continue → Skip to next iteration



Loops save time, reduce code and make your scripts smarter! 😊

Swipe →



10. FUNCTIONS



Functions help you organize code, reuse blocks of code and make your scripts more readable and maintainable.

Write once,
use many times!



① FUNCTION SYNTAX

```
function_name() {  
    # commands  
    return value # (optional)  
}
```

Function name
followed by ()

Commands to
be executed

Optional return
value (0-255)

② CALLING FUNCTIONS

- Call the function by its name.

```
greet           # Call greet function  
show_date       # Call show_date function  
add_numbers     # Call add_numbers function
```

③ PASSING PARAMETERS

- Pass values to functions as arguments.
- Use \$1, \$2, ... to access them.

```
welcome() {  
    echo "Hello, $1 $2!"  
}  
  
welcome "DevOps" "Engineer"
```

Inside function:

- \$1 → first argument
- \$2 → second argument
- \$# → number of arguments
- \$@ → all arguments
- * → all arguments (as one)

④ RETURN VALUES

- Use **return** to send a value back to the caller.
- Captured in \$? variable.

```
add() {  
    local sum=$(( $1 + $2 ))  
    return $sum # return sum  
}  
  
add 10 20  
echo "Sum = $?" # Output: Sum = 30
```

Notes:

- Return value is an integer between 0-255.
- Suitable for status codes or small values.

⑤ LOCAL VARIABLES

- Variables inside a function are global by default.
- Use **local** keyword to make them local to the function.

```
my_func() {  
    local msg="Hello" # Local variable  
    count=10          # Global variable  
    echo "$msg"  
}  
  
my_func  
echo "$msg" # (not accessible)  
echo "$count" # 10 (accessible)
```

★ Key Points

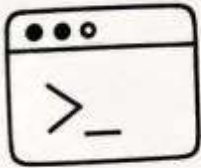
- ✓ Functions improve reusability
- ✓ Use meaningful names
- ✓ Keep functions short and focused
- ✓ Use local variables to avoid side effects

EXAMPLE: COMPLETE SCRIPT

```
#!/bin/bash  
  
greet() {  
    echo "Hello, $1!"  
}  
  
sum() {  
    local result=$(( $1 + $2 ))  
    return $result  
}  
  
greet "DevOps Engineer"  
sum 15 25  
echo "Total = $?" # Output: Total = 40
```



Functions make your scripts modular, clean and easy to maintain! 😊



11. FILE TEST OPERATORS



File test operators are used to check the status of files and directories.

Used with test command [] or [[]] in shell scripts!



Operator	Meaning	Example	Description
-f	File exists and is a regular file 	if [-f /etc/passwd]; then echo "File exists" fi	True if the specified path exists and is a regular file.
-d	File exists and is a directory 	if [-d /home/user]; then echo "Directory exists" fi	True if the specified path exists and is a directory.
-e	File or directory exists 	if [-e /tmp/data]; then echo "Exists" fi	True if the specified path exists (file or directory).
-r	File is readable 	if [-r file.txt]; then echo "Readable" fi	True if the file exists and has read permission.
-w	File is writable 	if [-w file.txt]; then echo "Writable" fi	True if the file exists and has write permission.
-x	File is executable 	if [-x script.sh]; then echo "Executable" fi	True if the file exists and has execute permission.
-s	File exists and has size > 0 	if [-s file.txt]; then echo "Not empty" fi	True if the file exists and has a size greater than zero.
-L	File is a symbolic link 	if [-L link]; then echo "Symlink" fi	True if the specified path is a symbolic link.

Example Script

```
#!/bin/bash
FILE="test.txt"
if [ -f "$FILE" ]; then
    echo "$FILE is a regular file."
elif [ -d "$FILE" ]; then
    echo "$FILE is a directory."
else
    echo "$FILE does not exist."
fi
```

Tip

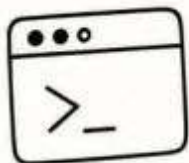
Combine operators for advanced checks!

Example:

```
if [ -f file.txt ] && [ -r file.txt ]; then
    echo "File exists and is readable"
fi
```



File test operators help you make your scripts smarter and more reliable! 😊



12. FILE OPERATIONS



File operations are used to create, copy, move, delete, and manage files and directories.

Master file operations to handle files like a pro! 😊

Command	Meaning	Syntax	Example	Description
<code>touch</code> 	Create an empty file or update the timestamp.	<code>touch filename</code>	<code>touch demo.txt</code> # creates demo.txt	Creates a new empty file or updates the timestamp of an existing file.
<code>cp</code> 	Copy files or directories.	<code>cp source destination</code> <code>cp -r source_dir dest_dir</code>	<code>cp file.txt backup.txt</code> <code>cp -r dir1 dir2</code> # copy file or directory	Copies source file or directory to destination.
<code>mv</code> 	Move or rename files/directories.	<code>mv source destination</code>	<code>mv oldname.txt newname.txt</code> <code>mv file.txt /home/user/</code> # move or rename	Moves a file/directory or renames a file.
<code>rm</code> 	Remove (delete) files or directories.	<code>rm filename</code> <code>rm -r directory</code>	<code>rm file.txt</code> <code>rm -r dir1</code> # delete file or directory	Deletes files or directories permanently. (Use with caution!)
<code>mkdir</code> 	Create a new directory.	<code>mkdir dirname</code> <code>mkdir -p dir1/dir2/dir3</code>	<code>mkdir mydir</code> <code>mkdir -p a/b/c</code> # create directories	Creates a new directory. -p creates parent directories if needed.
<code>rmdir</code> 	Remove an empty directory.	<code>rmdir dirname</code>	<code>rmdir mydir</code> # remove empty directory	Removes an empty directory.
<code>cat</code> 	Display the content of a file.	<code>cat filename</code> <code>cat file1 file2</code>	<code>cat file.txt</code> # display content <code>cat a.txt b.txt</code> # display multiple files	Displays file content on the terminal.
<code>tee</code> 	Read from standard input and write to a file and terminal.	<code>tee filename</code> <code>tee -a filename</code> # -a to append	<code>echo "Hello" tee out.txt</code> <code>echo "World" tee -a out.txt</code> # write/append to file & terminal	Writes input to a file and also displays it on the terminal.

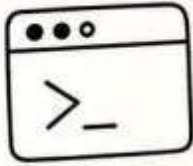


Tips:

- Use `rm -r` with caution - files deleted cannot be recovered easily.
- Use **tab completion** to save time while typing file or directory names.



Efficient file operations make your scripts powerful and reliable! 😊



13. EXIT STATUS



Exit status (or exit code) indicates whether a command or script executed successfully or failed.

Every command returns an exit status to the shell!



1 EXIT CODES

- In Unix/Linux, every command returns an exit code.
- Exit code 0 means success.
- Any **non-zero** value means failure.

Exit Code	Meaning
0	Success
1	General error
2	Misuse of shell builtins
126	Command invoked cannot execute
127	Command not found
128+N	Command terminated by signal N

2 CHECK EXIT STATUS

- Use `echo $?` to check the exit status of the last command.

```
ls /etc/passwd      # command
echo $?             # check exit status

cat /no_such_file    # this will fail
echo $?             # check exit status
```

\$? holds the exit status of the last executed command.



Example Output:

```
/etc/passwd
0
cat: /no_such_file: No such file or directory
1
```

3 EXIT IN SCRIPTS

- Use `exit` to terminate the script with a status.
- It is a good practice to exit with meaningful codes.

```
#!/bin/bash
if [ -f /etc/passwd ]; then
    echo "File exists."
    exit 0      # success
else
    echo "File not found!"
    exit 1      # failure
fi
```

exit 0 → Success ✓
exit 1 → Failure ✗

4 ERROR HANDLING

- Check exit status and handle errors gracefully.

```
command_that_may_fail
status=$?
if [ $status -ne 0 ]; then
    echo "Error: Command failed with exit code $status"
    exit $status    # exit with same error code
else
    echo "Command executed successfully."
fi
```

Always check and handle errors!



COMMON EXIT CODES

- | | |
|----------------------|-----------------------------------|
| • 0 Success | • 127 Command not found |
| • 1 General error | • 128+N Terminated by signal N |
| • 2 Incorrect usage | • 130 Script interrupted (Ctrl+C) |
| • 126 Cannot execute | • 137 Killed (SIGKILL) |

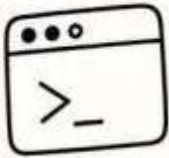


BEST PRACTICES

- ✓ Use meaningful exit codes in your scripts.
- ✓ Always check the exit status of critical commands.
- ✓ Propagate the error using `exit $status`.



Good scripts don't just run - they detect problems and respond the right way! 😊



14. INPUT/OUTPUT REDIRECTION



Input/Output redirection allows you to control where the input comes from and where the output goes.

Redirection makes your scripts flexible and powerful! 😊

Operator	Name	Description	Example	Explanation
>	Output Redirection	Redirects standard output (stdout) to a file. (overwrites the file)	ls -l > list.txt echo "Hello" > file.txt	Output is written to the file. If the file exists, it will be overwritten.
>>	Append Redirection	Redirects standard output (stdout) to a file. (appends to the file)	ls -l >> list.txt echo "Hello" >> file.txt	Output is appended to the existing file.
<	Input Redirection	Takes input from a file instead of keyboard.	wc -l < file.txt sort < names.txt	The command reads input from the file.
2>	Error Redirection	Redirects standard error (stderr) to a file. (overwrites the file)	ls /invalid 2> error.txt	Errors are written to the file. If the file exists, it will be overwritten.
2>>	Append Error Redirection	Redirects standard error (stderr) to a file. (appends to the file)	ls /invalid 2>> error.txt	Errors are appended to the existing file.
&>	Redirect Both Output & Error	Redirects both stdout and stderr to a file. (overwrites the file)	command &> output.txt	Both output and errors are written to the file.
/dev/null	Null Device	Discards all output. (redirect output to nowhere)	command > /dev/null command 2> /dev/null command &> /dev/null	Output and/or errors are discarded and not shown anywhere.

EXAMPLES

Overwrite output to a file
ls -l > files.txt

Append output to a file
echo "Hello" >> files.txt

Read input from a file
sort < names.txt

Redirect errors to a file
grep "xyz" file.txt 2> err.log

Redirect both output and errors
command &> all.log

Discard all output
command > /dev/null
command 2> /dev/null
command &> /dev/null

STANDARD STREAMS

0 → stdin (Standard Input)
1 → stdout (Standard Output)
2 → stderr (Standard Error)

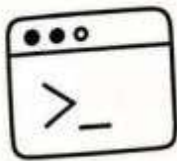


BEST PRACTICES



- ✓ Always handle errors in scripts using 2> or &>.
- ✓ Use >> to keep logs and history.
- ✓ Use /dev/null to keep the output clean when not needed. 😊

Always check your output and error logs while debugging scripts. It makes troubleshooting easier! 😊



15. PIPES & FILTERS



Pipes (|) and filters allow you to combine commands to process and analyze text efficiently.

Pipes make your commands powerful and your output meaningful! 😊

Command	Purpose	Syntax	Example	Explanation
	Pipe Passes output of one command as input to another.	command1 command2	ls -l less # view output page by page	The output of ls -l is passed to less command.
 grep	Search for patterns in text.	grep [options] pattern [file]	grep "error" /var/log/syslog # search for 'error' in file	Prints lines that match the pattern.
 awk	Pattern scanning and processing language.	awk 'pattern { action }' file	awk '{print \$1, \$3}' data.txt # print 1st and 3rd columns	Processes text by columns or patterns.
 sed	Stream editor for filtering and transforming text.	sed 's/old/new/g' file	sed 's/error/ERROR/g' log.txt # replace 'error' with 'ERROR'	Searches and replaces text in a stream.
 cut	Remove sections from each line of files.	cut -d 'delim' -f N file	cut -d ',' -f1,3 data.csv # print 1st and 3rd fields	Cuts fields/columns from the input.
 sort	Sort lines of text alphabetically or numerically.	sort [options] file	sort -n numbers.txt # sort numbers numerically	Sorts the lines in ascending order.
 uniq	Report or omit repeated lines.	uniq [options] file	sort names.txt uniq # remove duplicate lines	Removes consecutive duplicate lines.
 wc	Print newline, word, and byte counts.	wc [options] file	wc -l file.txt # count number of lines	Counts lines, words, and characters.

EXAMPLE WORKFLOW

```
cat access.log | grep "GET" | awk '{print $7}' |  
sort | uniq -c | sort -nr | head -5  
# 1. Read file  
# 2. Filter lines with GET  
# 3. Extract 7th column (URL)  
# 4. Sort URLs  
# 5. Remove duplicates and count  
# 6. Sort by count (desc)  
# 7. Show top 5 results
```

Why Use Pipes & Filters?

- ✓ Work with large data efficiently
- ✓ Combine small commands to do big tasks
- ✓ Save time and automate text processing 😊

QUICK TIP

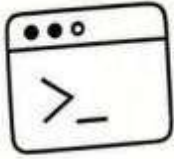


Use pipes to break down complex tasks into simple steps.

Example:

```
command1 | command2 | command3
```

Each command does one job! 😊



16. SPECIAL VARIABLES



Special variables in shell scripts store important information about the script, its arguments, and the last executed commands.

These variables are read-only and set by the shell!



Variable	Meaning	Example	Explanation
\$0	Name of the script.	\$0 # Output: ./myscript.sh	Returns the name of the script being executed.
\$1	First argument passed to the script.	\$1 # If script.sh hello world # \$1 = hello	Returns the first positional parameter.
\$#	Number of arguments passed to the script.	\$# # If script.sh a b c # \$# = 3	Returns the total count of arguments.
\$@	All arguments as separate strings.	\$@ # "\$@" = "a" "b" "c"	Expands to all arguments separately.
\$*	All arguments as a single string.	\$* # "\$*" = "a b c"	Expands to all arguments as one single string.
\$\$	Process ID of the current shell.	\$\$ # Output: 12345	Returns the PID of the current script process.
\$?	Exit status of the last executed command.	ls /no_such_file echo \$? # Output: 2	Returns the exit code of the last command.
\$!	Process ID of the last background command.	sleep 10 & echo \$! # Output: 67890	Returns the PID of the last command run in background.

Example Script

```
#!/bin/bash
echo "Script name: $0"
echo "First argument: $1"
echo "Total arguments: $#"
```

echo "All args (\$@): \$@"

echo "All args (\$*): \$*"

echo "PID of this script: \$\$"

ls /no_such_file

echo "Exit status: \$?"

sleep 5 &

echo "Last BG PID: \$!"

Run like this:

./script.sh apple banana cherry

RUN EXAMPLE

```
$ ./script.sh apple banana cherry
```

- **\$0** = ./script.sh
- **\$1** = apple
- **\$#** = 3
- **\$@** = apple banana cherry (as separate strings)
- **\$*** = apple banana cherry (as single string)
- **\$\$** = 12345 (example PID)
- **\$?** = 0 (if last command succeeded)
- **\$!** = 67890 (example background PID)

QUICK TIP

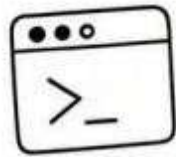
Use these variables to make your scripts dynamic, flexible, and more powerful!



IMPORTANT NOTES

- ✓ All special variables start with \$.
- ✓ They are read-only.
- ✓ \$@ and \$* behave differently in quotes.
- ✓ Use \$? to check errors and handle them in your scripts.





17. USEFUL SHELL COMMANDS



These commands help you display text, take input, perform operations, and manipulate files and strings efficiently in shell scripts.

Master these commands and your shell scripts will become powerful! 😊

Command	Purpose	Syntax	Example	Output/Explanation
echo	Display text/string on the terminal.	<code>echo [string]</code> <code>echo "text"</code>	<code>echo "Hello World"</code>	Hello World Prints the given text.
read	Read input from the user.	<code>read variable</code> <code>read -p "Prompt" variable</code>	<code>read -p "Enter name: " name</code> <code>echo "Hi \$name"</code>	Reads user input and stores in the variable.
printf	Display formatted text.	<code>printf "format" value</code>	<code>printf "Name: %s\n" "Rani"</code>	Name: Rani More control over formatting than echo.
expr	Evaluate expressions (arithmetic, string, etc.)	<code>expr expression</code>	<code>expr 5 + 3</code> <code>expr length "hello"</code>	8 5 Performs operations.
test	Check conditions (files, strings, numbers).	<code>test condition</code> or <code>[condition]</code>	<code>test -f file.txt</code> <code>["\$a" -eq 5]</code>	Returns true (0) if condition is true, else false (1).
basename	Remove directory path from a filename.	<code>basename [path]</code>	<code>basename /home/user/file.txt</code>	file.txt Returns the file name.
dirname	Remove filename from a path.	<code>dirname [path]</code>	<code>dirname /home/user/file.txt</code>	/home/user Returns the directory path.
xargs	Build and execute commands from stdin.	<code>command xargs [options]</code>	<code>echo "a b c" xargs echo</code> <code>ls *.txt xargs rm -f</code>	a b c Removes all .txt files. Constructs commands from standard input.



QUICK TIPS

- ✓ Use `echo` or `printf` to display messages.
- ✓ Use `read` to take user input.
- ✓ Use `test` or `[ ]` to make decisions in scripts.
- ✓ Use `basename` and `dirname` when working with file paths.
- ✓ Use `xargs` to pass output of one command as input to another.

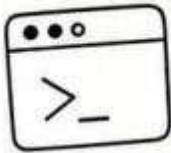


EXAMPLE COMBINATION

```
cat names.txt | grep "r" | sort | uniq -c | xargs
```

↓ ↓ ↓ ↓ ↓

Read file Filter lines with 'r' Sort lines Count unique lines Build & execute command



18. DEVOPS AUTOMATION EXAMPLES



Shell scripting is widely used in DevOps to automate repetitive tasks, monitor systems, and improve operational efficiency.

Automate today,
Deploy better,
Scale tomorrow! 😊

1	 HEALTH CHECK SCRIPT Check if a service or endpoint is up and running.	<pre>#!/bin/bash URL="http://localhost:8080/health" if curl -s --head --request GET "\$URL" grep -q "200 OK"; then echo "✅ Service is UP" else echo "❌ Service is DOWN" exit 1 fi</pre>	<pre># Checks HTTP # endpoint and # returns status</pre>
2	 DISK USAGE MONITORING Monitor disk usage and alert if usage exceeds a threshold.	<pre>#!/bin/bash THRESHOLD=80 USAGE=\$(df / awk 'NR==2 {print \$5}' tr -d '%') if ["\$USAGE" -ge "\$THRESHOLD"]; then echo "⚠️ Disk usage is \${USAGE}% - Exceeds threshold!" mail -s "Disk Alert" admin@example.com fi</pre>	<pre># Alerts admin # if disk usage # crosses limit</pre>
3	 BACKUP AUTOMATION Automate backups of important directories or databases.	<pre>#!/bin/bash BACKUP_DIR="/backups" DATE=\$(date +%F) tar -czf \$BACKUP_DIR/backup_\$DATE.tar.gz /data echo "✅ Backup completed: backup_\$DATE.tar.gz"</pre>	<pre># Creates compressed # backup with # date stamp</pre>
4	 LOG MONITORING Monitor log files for errors and send alert.	<pre>#!/bin/bash LOGFILE="/var/log/app.log" ERROR_COUNT=\$(grep -ci "error" \$LOGFILE) if ["\$ERROR_COUNT" -gt 0]; then echo "⚠️ Found \$ERROR_COUNT error(s) in log" mail -s "Log Alert" admin@example.com fi</pre>	<pre># Scans log file # for errors and # sends alert</pre>
5	 SERVICE RESTART Check service status and restart if it's not running.	<pre>#!/bin/bash SERVICE="nginx" if ! systemctl is-active --quiet \$SERVICE; then echo "⚙️ \$SERVICE is not running. Restarting..." systemctl restart \$SERVICE fi</pre>	<pre># Ensures service # is always # running</pre>
6	 CRON JOBS Schedule scripts to run automatically using cron.	<pre># Edit crontab -e 0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1</pre>	<pre># Runs backup # every day at # 2:00 AM</pre>
7	 DEPLOYMENT AUTOMATION Automate application deployment with git pull and service restart.	<pre>#!/bin/bash cd /opt/myapp exit 1 git pull origin main npm install --production systemctl restart myapp echo "🚀 Deployment completed successfully!"</pre>	<pre># Pulls latest code, # installs deps and # restarts app</pre>



BEST PRACTICES

- ✅ Use meaningful logs and alerts.
- ✅ Always handle errors and exceptions.
- ✅ Test scripts before automating in production.
- ✅ Use absolute paths in scripts.
- ✅ Secure sensitive data (use env vars or secrets).



COMMON TOOLS USED

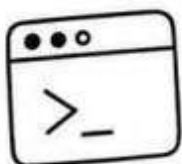
	cron	Scheduling tasks
	mail / mailx	Send email alerts
	systemctl	Manage services
	df, du, grep, awk	System monitoring
	git	Code deployment



**AUTOMATE EVERYTHING,
REPEAT NOTHING!**

Shell scripts are the
backbone of DevOps
automation.





19. SHELL SCRIPTING BEST PRACTICES



Following best practices makes your scripts robust, easy to read, easy to maintain, and less error-prone.

Write clean, reliable and maintainable shell scripts! 😊

#	Best Practice	Why It Matters	Example (Good)	Example (Bad)
1	 Use Meaningful Variable Names	Improves readability and maintainability.	<code>LOG_FILE="/var/log/app.log"</code> <code>BACKUP_DIR="/backups"</code>	<code>f="/var/log/app.log"</code> <code>d="/b"</code>
2	 Quote Variables	Prevents word splitting and globbing issues.	<code>echo "File is \$FILE"</code> <code>if [-f "\$FILE"]; then ...</code>	<code>echo File is \$FILE</code> <code>if [-f \$FILE]; then ...</code>
3	 Handle Errors	Helps detect issues early and avoid silent failures.	<code>if ! command; then</code> <code>echo "Error occurred"</code> <code>exit 1</code> <code>fi</code>	<code>command</code> <code>echo "Done"</code>
4	 Use Functions	Avoid repetition and make code modular.	<code>backup() {</code> <code>tar -czf backup.tar.gz /data</code> <code>}</code> <code>backup</code>	<code># Repeated code</code> <code>tar -czf backup1.tar.gz /data</code> <code>tar -czf backup2.tar.gz /data</code>
5	 Add Comments	Explains what the script does and why.	<code># Backup important data</code> <code>tar -czf backup.tar.gz /data</code>	<code>tar -czf backup.tar.gz /data</code>
6	 Validate Inputs	Prevents invalid input and unexpected behavior.	<code>if [-z "\$1"]; then</code> <code>echo "Usage: \$0 <dir>"</code> <code>exit 1</code> <code>fi</code>	<code>DIR=\$1</code> <code>cd \$DIR</code>
7	 Test Scripts	Find bugs early and ensure reliability.	<code>shellcheck script.sh</code> <code>sh -n script.sh</code> <code>./script.sh --test</code>	<code># No testing</code>
8	 Follow Consistent Formatting	Improves readability and team collaboration.	<code>if ["\$VAR" = "value"]; then</code> <code>echo "OK"</code> <code>else</code> <code>echo "Not OK"</code> <code>fi</code>	<code>if["\$VAR"="value"];then</code> <code>echo"OK";else</code> <code>echo"Not OK";fi</code>

GENERAL TIPS



- ✓ Use `set -e` to exit on errors.
- ✓ Use `set -u` to treat unset variables as errors.
- ✓ Use `set -o pipefail` to catch pipe errors.
- ✓ Keep scripts simple and focused.
- ✓ Use meaningful exit codes.
- ✓ Keep sensitive data out of scripts.



EXAMPLE: ROBUST SCRIPT STRUCTURE

```
#!/bin/bash
set -euo pipefail # Exit on error, unset var, pipe failures

# --- Config ---
LOG_FILE="/var/log/myapp.log"

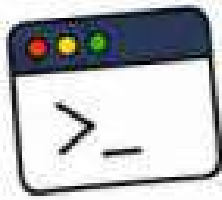
# --- Functions ---
log() { echo "[$(date +%F %T)] $1" >> "$LOG_FILE"; }

# --- Main ---
log "Script started"
# ... your code ...
log "Script completed successfully"
exit 0
```



Good Practices
= Reliable
Scripts!

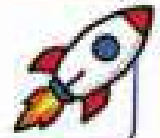




20. DEVOPS INTERVIEW QUESTIONS & CTA



Master these Shell Scripting concepts and you'll be one step ahead in your DevOps interviews!

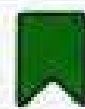


TOP INTERVIEW QUESTIONS

- 1  What is Shell Scripting?
- 2  Explain Shebang.
- 3  \$@ vs \$* Difference between \$@ and \$*.
- 4  What is \$?.
- 5  Explain for vs while.
- 6  How do you handle errors?
- 7  What are file test operators?
- 8  Explain pipes and redirection.
- 9  Explain input/output redirection.



SAVE THIS CAROUSEL
for interview preparation.



COMMENT "DOC"
to get the complete Shell Scripting guide.



This structure is ideal for a high-value educational carousel and can later be expanded into 20 detailed posts, one for each slide.

